

Raspberry Pi

22026

Juan Camilo Acevedo Bedoya | Facultad de Ingenierías

Arquitectura y Programación en Ensamblador de Raspberry Pi 3 y 4

El Raspberry Pi es una serie de computadoras de placa única que ha evolucionado significativamente en términos de rendimiento y capacidad. Estos pequeños pero potentes computadoras de placa única han revolucionado la educación en informática y la creación de proyectos DIY desde su lanzamiento en 2012.

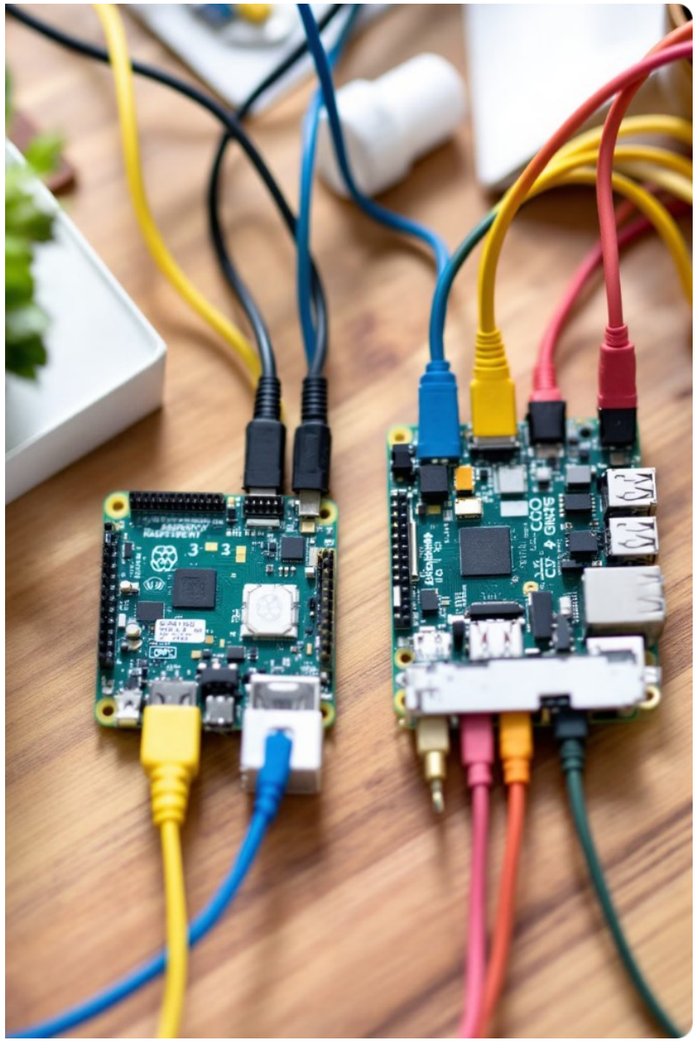
La Raspberry Pi es una serie de computadoras de placa única (SBC, por sus siglas en inglés: Single Board Computer) desarrolladas por la Raspberry Pi Foundation en el Reino Unido. Fue creada con el objetivo de fomentar el aprendizaje de la informática y la programación en niños y adultos a un costo accesible.

Antes del Raspberry Pi, los fundadores de la Raspberry Pi Foundation, en especial Eben Upton, notaron que había un gran declive en las habilidades de programación de los estudiantes que ingresaban a la Universidad de Cambridge para estudiar informática. Esto se debía a que, a diferencia de los años 80 y 90, los niños ya no tenían acceso a computadoras básicas programables como las BBC Micro, Commodore 64, o las Amiga, sino que crecían con dispositivos cerrados como consolas de videojuegos y PCs con sistemas operativos que no incentivaban la programación.

El objetivo de la fundación era crear un dispositivo barato, accesible y fácil de usar para permitir que más personas aprendieran a programar. Así nació el primer Raspberry Pi en 2012.

Importancia del Raspberry Pi en la Computación

- **Educación:** Ha sido una herramienta clave para enseñar programación, electrónica y robótica en escuelas y universidades.
- **Internet de las Cosas (IoT):** Se usa ampliamente en automatización, domótica y monitoreo remoto.
- **Servidores y redes:** Muchos entusiastas lo usan como servidor de archivos, VPN o centro multimedia con Kodi.
- **Industria y ciencia:** Se usa en aplicaciones de visión por computadora, IA y procesamiento de datos.



Evolución de la Raspberry Pi

A lo largo de los años, el Raspberry Pi ha evolucionado en términos de hardware, capacidades de procesamiento y conectividad.

Raspberry Pi 1 (2012)

- Procesador: Broadcom BCM2835 (ARM11, 700 MHz, 32 bits)
- Memoria RAM: 256 MB (Model B), luego ampliada a 512 MB
- Puertos: 2x USB 2.0, HDMI, ranura SD
- Sistema Operativo: Raspbian basado en Debian

Impacto: Fue un gran éxito, vendiéndose más de un millón de unidades en el primer año. A pesar de su modesto rendimiento, permitió a muchas personas aprender sobre Linux, Python y electrónica.

Evolución de la Raspberry Pi

Raspberry Pi 2 (2015)

- Procesador: Broadcom BCM2836 (ARM Cortex-A7, 900 MHz, 4 núcleos, 32 bits)
- Memoria RAM: 1 GB
- Conectividad mejorada con más puertos USB

Mejoras: Mucho más rápido que el Pi 1, con un procesador de cuatro núcleos, lo que permitió ejecutar sistemas operativos más exigentes como Windows 10 IoT Core.

Raspberry Pi 3 (2016)

- Procesador: Broadcom BCM2837 (ARM Cortex-A53, 1.2 GHz, 64 bits)
- Memoria RAM: 1 GB
- Conectividad: Primer Raspberry Pi con Wi-Fi y Bluetooth integrados
- Soporte inicial para arquitectura de 64 bits (aunque el sistema operativo por defecto seguía siendo de 32 bits)

Impacto: Se convirtió en una excelente opción para proyectos de IoT y robótica, gracias a su conectividad inalámbrica integrada.

Evolución de la Raspberry Pi

Raspberry Pi 4 (2019)

- Procesador: Broadcom BCM2711 (ARM Cortex-A72, 1.5 GHz, 64 bits, 4 núcleos)
- Memoria RAM: 2 GB, 4 GB, y luego 8 GB (dependiendo de la versión)
- Puertos: USB 3.0, Gigabit Ethernet, 2x HDMI microSoporte nativo para 4KMás capacidad de procesamiento y almacenamiento

Innovación: Fue el primer modelo con suficiente potencia para reemplazar a una computadora de escritorio básica, lo que lo convirtió en una opción viable para más aplicaciones avanzadas, incluyendo inteligencia artificial y servidores.

Raspberry Pi 5 (2023)

- Procesador: Broadcom BCM2712 (ARM Cortex-A76, 2.4 GHz, 4 núcleos, 64 bits)
- Memoria RAM: 4 GB y 8 GB LPDDR4X
- GPU mejorada con soporte para OpenGL ES 3.1 y Vulkan 1.2
- Puerto PCIe para expansión de almacenamiento o tarjetas aceleradoras

Impacto: Este modelo aumentó drásticamente el rendimiento en comparación con el Pi 4, gracias a su arquitectura más moderna y su mejor gestión de energía.

Arquitectura del Procesador Raspberry Pi 3

1

SoC Broadcom BCM2837

El Raspberry Pi 3 utiliza un System-on-Chip (SoC) Broadcom BCM2837 que integra CPU, GPU, RAM y otros componentes en un único chip. Este diseño permite una mayor eficiencia energética y un tamaño reducido, ideal para dispositivos embebidos.

2

ARM Cortex-A53 de 64 bits

Incorpora cuatro núcleos ARM Cortex-A53 que implementan la arquitectura ARMv8-A. Es capaz de ejecutar tanto código de 32 bits (AArch32) como de 64 bits (AArch64), ofreciendo retrocompatibilidad con software anterior.

3

Frecuencia de 1.2 GHz

Opera a una frecuencia base de 1.2 GHz, que posteriormente fue mejorada a 1.4 GHz en el modelo 3B+. Incluye memoria caché L1 de 32KB para instrucciones y 32KB para datos por núcleo, y una caché L2 compartida de 512KB.

Arquitectura del Procesador Raspberry Pi 4

SoC Broadcom BCM2711

El Raspberry Pi 4 presenta un salto significativo con el SoC Broadcom BCM2711. Este chip integra componentes más potentes manteniendo el factor de forma compacto, pero con un rendimiento considerablemente superior al de su predecesor.

ARM Cortex-A72 de 64 bits

Dispone de cuatro núcleos ARM Cortex-A72 que implementan la arquitectura ARMv8-A. Este procesador es significativamente más potente que el Cortex-A53, ofreciendo hasta tres veces más rendimiento por núcleo a la misma frecuencia.

Frecuencia de 1.5 GHz

Funciona a 1.5 GHz con cachés L1 de 48KB para instrucciones y 32KB para datos por núcleo, y una caché L2 compartida de 1MB. Incluye además mejoras en la predicción de saltos y en la ejecución fuera de orden.

Diferencias principales

Característica	Raspberry Pi 3 (BCM2837)	Raspberry Pi 4 (BCM2711)
Arquitectura	ARMv8-A (Cortex-A53)	ARMv8-A (Cortex-A72)
Frecuencia	1.2 GHz	1.5 GHz
Núcleos	4	4
Caché L1	32 KB	48 KB
Caché L2	512 KB	1 MB
Soporte 64 bits	Limitado en Raspbian	Completo

El **Cortex-A72** en el Raspberry Pi 4 es significativamente más rápido gracias a su microarquitectura mejorada, una predicción de saltos más avanzada y mejor rendimiento en operaciones de coma flotante.

Set de Instrucciones ARM

ARM vs Thumb

ARM ofrece dos conjuntos de instrucciones principales: ARM (instrucciones de 32 bits) y Thumb (instrucciones de 16 bits). Thumb optimiza la densidad de código, sacrificando algunas capacidades pero reduciendo el tamaño del programa y mejorando el rendimiento en sistemas con buses de memoria estrechos.

Compatibilidad con versiones anteriores

La arquitectura ARMv8-A presente en las Raspberry Pi 3 y 4 mantiene compatibilidad con los modos de ejecución de 32 bits de ARMv7-A. Esto permite ejecutar aplicaciones y sistemas operativos diseñados para versiones anteriores sin modificaciones.

Ventajas del set de instrucciones ARM

El conjunto de instrucciones ARM destaca por su simplicidad, regularidad y eficiencia energética. Su diseño RISC (Reduced Instruction Set Computing) favorece implementaciones hardware más simples, veloces y con menor consumo energético.

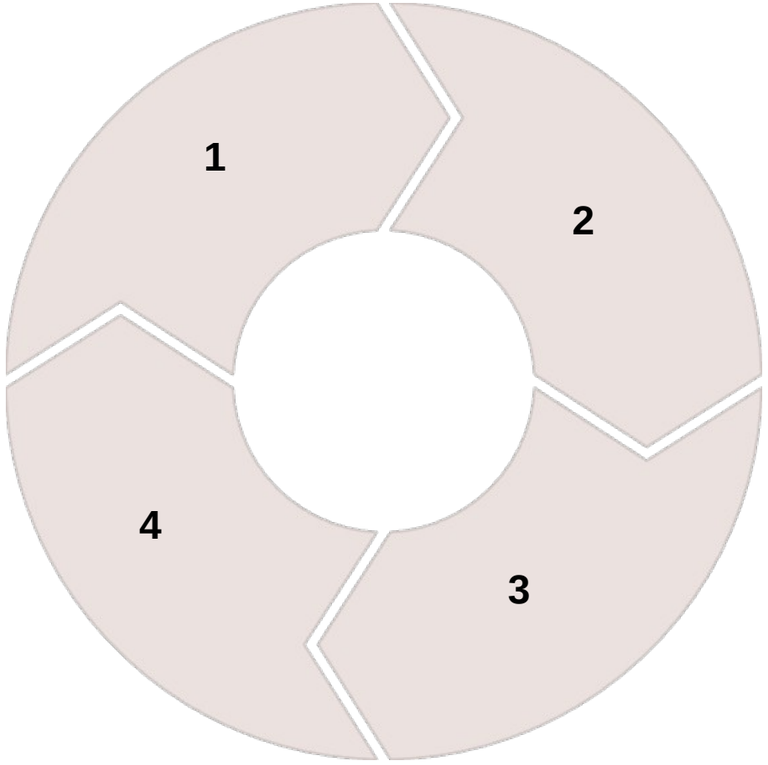
Registros en Raspberry Pi 3 y 4

Registros de Propósito General

31 registros de 64 bits (X0-X30) en modo AArch64. En modo AArch32, se accede a 16 registros de 32 bits (R0-R15).

Registros SIMD

32 registros vectoriales de 128 bits (V0-V31) para instrucciones NEON, que permiten operaciones en paralelo sobre múltiples datos.



Registro de Estado

PSTATE en AArch64 y CPSR en AArch32, que almacenan flags de condición y controlan modos de ejecución.

Registros Especiales

SP (Stack Pointer), PC (Program Counter), LR (Link Register, X30), y registros del sistema para control de MMU, excepciones y características avanzadas.

Registros en Raspberry Pi 3 y 4

El conjunto de registros en ARM sigue un modelo generalizado. En ARMv8-A, existen registros de **64 bits** cuando se usa en modo AArch64 y registros de **32 bits** cuando se usa en modo AArch32.

Registros Generales

- **X0-X30** (cuando se usa en modo AArch64): Registros de propósito general de 64 bits.
- **W0-W30** (cuando se usa en modo AArch32): Versiones de 32 bits de los registros X.
- **XZR/WZR**: Registro cero.
- **SP (Stack Pointer)**: Puntero de pila.
- **LR (Link Register)**: Guarda la dirección de retorno en llamadas a subrutinas.
- **PC (Program Counter)**: Contiene la dirección de la siguiente instrucción.

Registros Especiales

- **PSTATE**: Guarda el estado del procesador (banderas de condición).
- **FPSR y FPCR**: Registros para operaciones de coma flotante.

Modos de Operación

Modo de Ejecución AArch64

Es el modo nativo de 64 bits de la arquitectura ARMv8-A. Proporciona acceso al espacio de direcciones de 64 bits y a los 31 registros de propósito general de 64 bits. En este modo, las instrucciones tienen un tamaño fijo de 32 bits, simplifican la decodificación y mejoran el rendimiento.

Cambios entre Modos

Las transiciones entre modos AArch64 y AArch32 son controladas por el sistema operativo y solo pueden realizarse en ciertos puntos de excepción. Linux en Raspberry Pi puede ejecutar aplicaciones de 32 y 64 bits simultáneamente, realizando los cambios de contexto necesarios de forma transparente.

1

2

Modo de Ejecución AArch32

Es el modo de compatibilidad de 32 bits que emula la arquitectura ARMv7-A. Utiliza los registros R0-R15 de 32 bits y puede ejecutar instrucciones ARM de 32 bits o Thumb de 16/32 bits. Este modo es crucial para la compatibilidad con sistemas operativos y aplicaciones preexistentes.

3

Formato de Instrucciones ARM

1

Instrucciones ARM (32 bits)

En el modo ARM tradicional, todas las instrucciones tienen un tamaño fijo de 32 bits (4 bytes). Esto permite una decodificación más sencilla pero requiere más memoria para almacenar el código del programa. Cada instrucción puede realizar operaciones más complejas debido a la mayor cantidad de bits disponibles para codificarlas.

2

Instrucciones Thumb (16/32 bits)

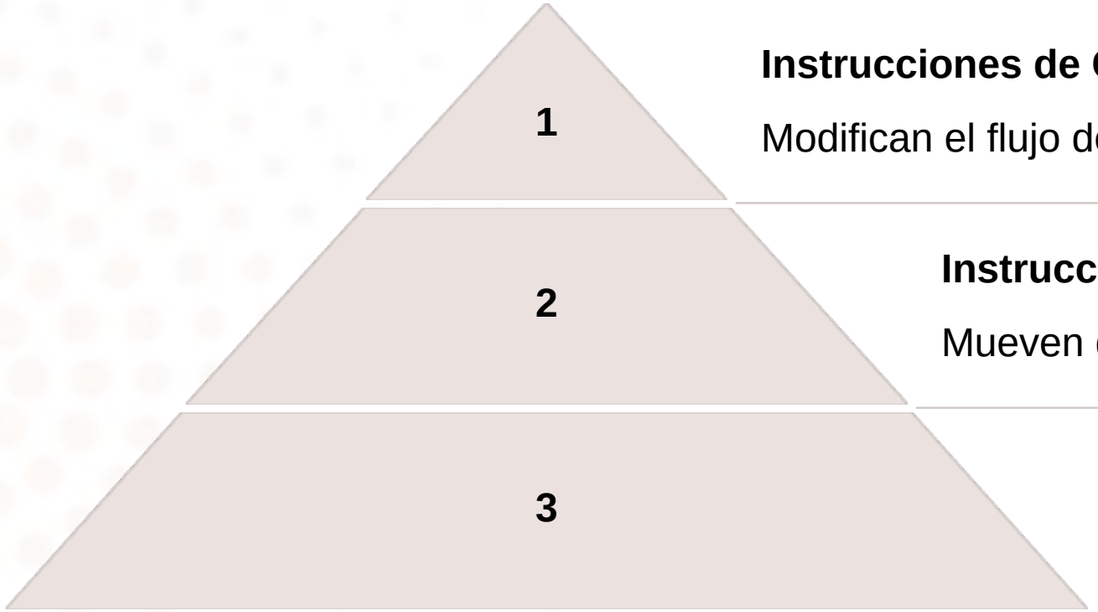
El conjunto de instrucciones Thumb utiliza instrucciones de 16 bits (Thumb) o de 32 bits (Thumb-2) según la operación. Esto proporciona una mayor densidad de código, reduciendo el tamaño de los programas hasta en un 30%. Es especialmente ventajoso en sistemas con buses de memoria de 16 bits o con restricciones de memoria.

3

Campos Comunes

Las instrucciones ARM típicamente incluyen: código de operación (OPCODE), registros fuente y destino, valores inmediatos, banderas de actualización de condición, y en algunas instrucciones, bits para modos de direccionamiento especiales o para operaciones específicas como desplazamientos.

Tipos de Instrucciones



Instrucciones de Control de Flujo

Modifican el flujo de ejecución del programa

Instrucciones de Transferencia de Datos

Mueven datos entre memoria y registros

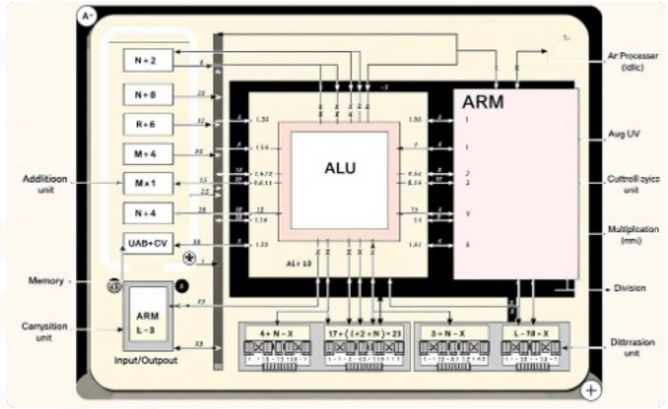
Instrucciones de Procesamiento de Datos

Realizan operaciones aritméticas y lógicas

La arquitectura ARM implementa un conjunto de instrucciones tipo RISC (Reduced Instruction Set Computing), caracterizado por su simplicidad y regularidad. Cada instrucción está optimizada para realizar una tarea específica de manera eficiente, siguiendo el principio de "una instrucción, una operación".

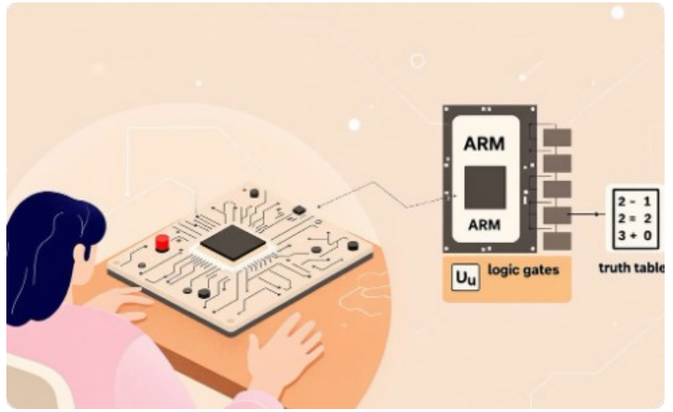
La mayoría de las instrucciones pueden ejecutarse condicionalmente según el estado de las banderas (N, Z, C, V) almacenadas en el registro de estado. Esta característica reduce la necesidad de instrucciones de salto, optimizando tanto el rendimiento como el tamaño del código.

Instrucciones de Procesamiento de Datos



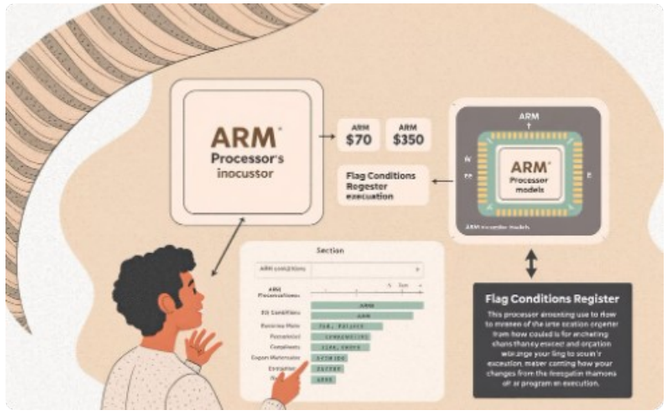
Operaciones Aritméticas

Incluyen ADD, SUB, MUL, DIV para suma, resta, multiplicación y división. También operaciones como ADC y SBC con acarreo. El formato general es: OP{S} Rd, Rn, Operando2, donde {S} activa la actualización de flags. El tamaño en bits depende del modo: 32 bits en ARM, 16/32 en Thumb.



Operaciones Lógicas

Comprenden AND, ORR, EOR (XOR), BIC (AND NOT) para operaciones a nivel de bit. Permiten manipulaciones bits específicos y construcción de máscaras. También incluyen operaciones de desplazamiento como LSL, LSR, ASR, ROR para desplazar bits a izquierda, derecha o rotar.



Instrucciones de Comparación

Las instrucciones CMP, CMN, TST y TEQ comparan valores o prueban bits sin guardar resultados, solo actualizando flags. Son fundamentales para tomas de decisiones en saltos condicionales y para implementar estructuras de control como if-else o bucles.

Instrucciones de Transferencia de Datos

Instrucciones Load y Store

La arquitectura ARM emplea un modelo de carga-almacenamiento (load-store), donde LDR carga datos de memoria a registros y STR almacena de registros a memoria. Existen variantes para diferentes tamaños: LDRB/STRB (byte), LDRH/STRH (halfword), LDR/STR (word) y LDRD/STRD (doubleword en ARMv8).

Modos de Direccionamiento

ARM soporta varios modos: inmediato (con offset constante), registro (usando valor en registro), pre-indexado (modificando dirección antes del acceso), post-indexado (modificando dirección después) y escalado (con desplazamientos). Esto permite acceso flexible a estructuras de datos complejas y arrays.

Transferencia Múltiple

Las instrucciones LDM/STM permiten cargar/almacenar múltiples registros con una sola instrucción, aumentando la eficiencia. Son especialmente útiles para guardar/restaurar contextos en llamadas a función o manejo de interrupciones, y para transferencias de bloques de datos.

Instrucciones de Control de Flujo

1	**Saltos** B (branch) para saltos incondicionales y B{cond} para condicionales
2	**Llamadas** BL (branch with link) para subrutinas, guardando dirección de retorno
3	**Retornos** RET/BX LR para volver de subrutinas usando el Link Register

Las instrucciones de control de flujo alteran la secuencia normal de ejecución del programa modificando el contenido del registro PC (Program Counter). El salto máximo posible está limitado por el campo de desplazamiento: ±16MB en modo ARM y ±1MB en Thumb.

Una característica distintiva de ARM es la ejecución condicional, que permite que casi cualquier instrucción se ejecute solo si se cumple una condición específica, basada en los flags N (negativo), Z (cero), C (acarreo) y V (desbordamiento). Esto reduce la necesidad de saltos y mejora el rendimiento en código con muchas decisiones.

Estructura de un Programa en Ensamblador

Secciones del Programa

.text para el código ejecutable, .data para datos inicializados, .bss para datos no inicializados, y .rodata para constantes. Esta organización permite al sistema operativo aplicar permisos adecuados: la sección .text es generalmente de solo lectura y ejecutable, mientras que .data es de lectura/escritura.

1

Directivas del Ensamblador

Comandos que comienzan con punto (.) como .global, .align, .word, etc. No generan código máquina sino que instruyen al ensamblador. Por ejemplo, .global marca símbolos visibles al enlazador, .align asegura alineación de memoria, y .include permite incorporar otros archivos.

2

Etiquetas y Comentarios

Las etiquetas identifican posiciones en el código, terminando con dos puntos. Los comentarios comienzan con @ o // y son ignorados por el ensamblador. Una buena práctica es documentar cada sección, explicando los parámetros de entrada/salida de las funciones y la lógica utilizada.

3

Ejemplo de Código: Hola Mundo

```
/* hola_mundo.s */
.section .data
mensaje:
    .asciz "Hola, Mundo de Raspberry Pi!\n"
    .align 2

.section .text
.global main
main:
    mov x0, #1          // 1 = stdout
    ldr x1, =mensaje    // puntero al mensaje
    mov x2, #26         // longitud
    mov x8, #64         // syscall write
    svc #0              // llamada al sistema

    mov x0, #0          // código de salida
    mov x8, #93         // syscall exit
    svc #0              // llamada al sistema
```

Este sencillo programa muestra un mensaje en la consola. Comienza definiendo una sección .data con la cadena "Hola, Mundo de Raspberry Pi!" almacenada como secuencia ASCII terminada en cero (.asciz).

En la sección .text, se define main como punto de entrada (.global main). Preparamos los registros para la llamada al sistema write: x0=1 (stdout), x1=puntero al mensaje, x2=26 (longitud). La llamada al sistema se invoca con SVC #0 tras cargar su número en x8.

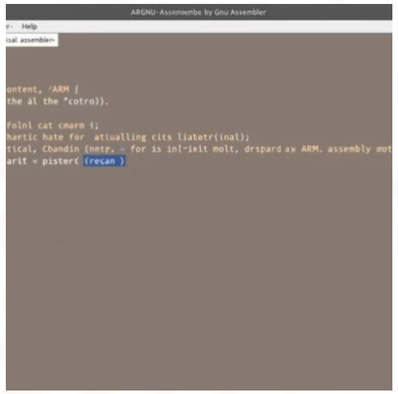
Finalmente, se realiza una llamada a exit con código de retorno 0.

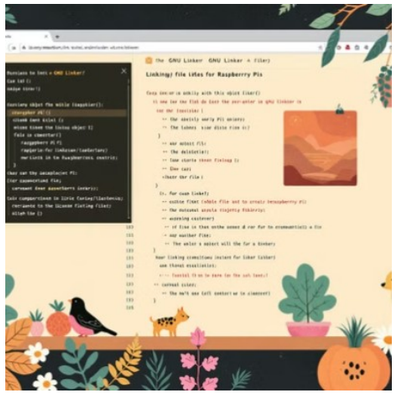
Para ensamblar este código:
as -o hola_mundo.o hola_mundo.s

Para enlazar:
ld -o hola_mundo hola_mundo.o

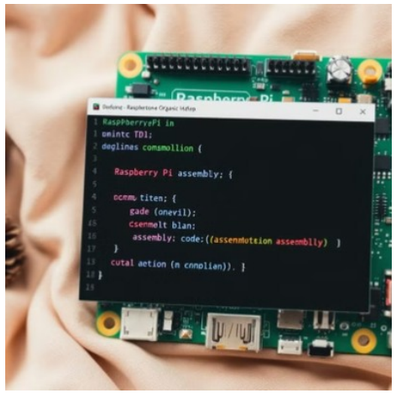
Para ejecutar:
./hola_mundo

Programación en Ensamblador: Herramientas









Para programar en ensamblador en Raspberry Pi, se necesitan varias herramientas esenciales. El GNU Assembler (as) traduce código ensamblador a código objeto, interpretando las instrucciones ARM y sus mnemónicos. Es parte de la suite GNU Binutils y soporta completamente la sintaxis ARM.

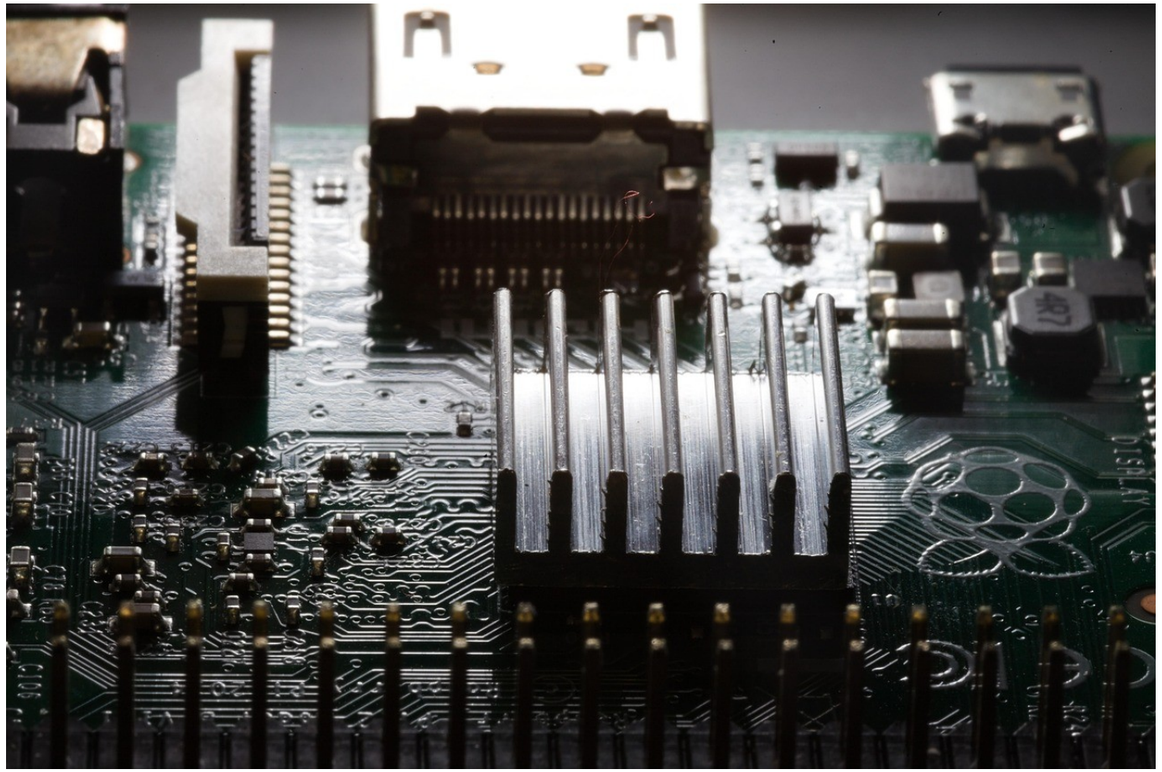
El GNU Linker (ld) combina múltiples archivos objeto en un ejecutable, resuelve referencias entre módulos y organiza el código según las secciones definidas. Para depuración, GDB (GNU Debugger) permite ejecutar programas paso a paso y examinar registros y memoria durante la ejecución.

Como entornos de desarrollo, se puede utilizar editores como Visual Studio Code con extensiones para ARM, o IDEs completos como Eclipse con plugins adecuados. Estas herramientas proporcionan resaltado de sintaxis, autocompletado y otras facilidades que mejoran la productividad.

Herramientas para el Desarrollo en Ensamblador

Para programar en ensamblador en Raspberry Pi, se pueden usar las siguientes herramientas:

1. **GNU Assembler (as)**: Ensamblador usado con GCC.
2. **GDB (GNU Debugger)**: Depurador para ARM.
3. **QEMU**: Emulador para probar código de ensamblador.
4. **LLVM/Clang**: Alternativa a GCC con soporte para ensamblador ARM.



Acceso a GPIO en Ensamblador

El **Raspberry Pi 3 y 4** tienen un conjunto de pines **GPIO (General Purpose Input/Output)** que permiten la interacción con el mundo exterior mediante la activación o lectura de señales digitales.

Mapa de memoria de GPIO

Los pines GPIO en Raspberry Pi se controlan a través de registros mapeados en memoria. La dirección base del controlador GPIO es 0x3F200000 en Pi 3 y 0xFE200000 en Pi 4. Cada pin tiene registros asociados para configuración, estado y operaciones de set/clear.

Configuración de pines

Para configurar un pin como entrada o salida, se debe escribir en los registros GPFSEL (Function Select). Cada pin tiene 3 bits de configuración, permitiendo seleccionar su función: entrada, salida o función alternativa (UART, SPI, etc.).

Lectura y escritura

La escritura se realiza usando registros GPSET (para poner a 1) o GPCLR (para poner a 0). La lectura del estado actual se hace a través del registro GPLEV. Para mayor eficiencia, estas operaciones permiten manipular múltiples pines con una sola instrucción.

Introducción a los GPIO

Los pines GPIO en Raspberry Pi están controlados por registros de memoria mapeados. Para interactuar con ellos, necesitamos acceder a direcciones específicas en la memoria que corresponden a los registros de control de GPIO.

Los registros de GPIO están en la **dirección base de GPIO (GPIO_BASE)**, que se calcula como:

GPIO_BASE = PERIPHERAL_BASE + 0x200000

Modelo	Dirección Base de Periféricos	Dirección Base de GPIO
Raspberry Pi 3	0x3F000000	0x3F200000
Raspberry Pi 4	0xFE000000	0xFE200000

Registros Importantes para GPIO

Para controlar los GPIO en ensamblador, debemos escribir en registros específicos:

Registro	Dirección (relativa a GPIO_BASE)	Función
GPFSELn	0x00 - 0x14	Configuración de modo (entrada/salida)
GPSETn	0x1C - 0x20	Activación del GPIO (poner en "1")
GPCLRn	0x28 - 0x2C	Apagar el GPIO (poner en "0")
GPLEVn	0x34 - 0x38	Leer el estado del GPIO

Cada GPIO está asociado con un bit dentro de estos registros.

Ejemplo: Encender y apagar un LED en el GPIO 17 (RPi 3 y 4)

```
.section .data
.equ PERIPHERAL_BASE, 0xFE000000 // Base de periféricos para Raspberry Pi 4
.equ GPIO_BASE, PERIPHERAL_BASE + 0x200000 // Base de GPIO
.equ GPFSEL1, GPIO_BASE + 0x04 // Registro de selección de función para GPIO 10-19
.equ GPSET0, GPIO_BASE + 0x1C // Registro para activar GPIOs
.equ GPCLR0, GPIO_BASE + 0x28 // Registro para apagar GPIOs
.equ GPFSEL17_OUTPUT, 1 << 21 // Configura GPIO 17 como salida (bits 21-23)
.equ GPIO17, 1 << 17 // Máscara para GPIO 17

.section .text
.global _start

_start:
        // 1. Configurar GPIO17 como salida
        LDR X0, =GPFSEL1 // Cargar la dirección de GPFSEL1
        LDR X1, [X0] // Leer el contenido actual del registro
        BIC X1, X1, #(7 << 21) // Limpiar bits 21-23 para GPIO17
        ORR X1, X1, #GPFSEL17_OUTPUT // Configurar GPIO17 como salida
        STR X1, [X0] // Escribir en el registro
loop:
        // 2. Encender GPIO17
        LDR X0, =GPSET0
        MOV X1, #GPIO17 STR X1, [X0]
        // 3. Esperar un tiempo (delay simple)
        MOV X2, #500000
delay_on:
        SUBS X2, X2, #1
        BNE delay_on
        // 4. Apagar GPIO17
        LDR X0, =GPCLR0
        MOV X1, #GPIO17
        STR X1, [X0]
        // 5. Esperar un tiempo (delay simple)
        MOV X2, #5000000
delay_off:
        SUBS X2, X2, #
         BNE delay_off
        B loop // Repetir el ciclo
```

Ejemplo: Encender y apagar un LED en el GPIO 17 (RPi 3 y 4)

Compilar el código

Guardar el código como gpio_led.s y compilarlo con:

as -o gpio_led.o gpio_led.s ld -o gpio_led gpio_led.o

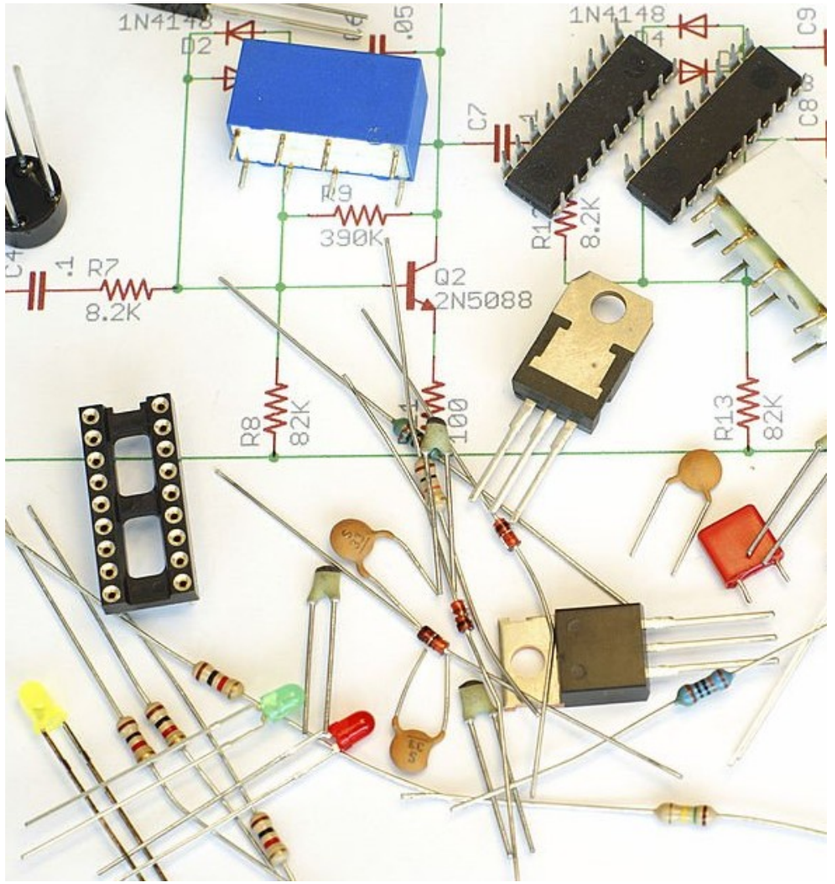
Ejecutar el código

Ejecutar el código en Raspberry Pi con:

sudo ./gpio_led

Conectar un LED

Para probar el código físicamente se conectar un LED al GPIO 17 (pata larga del LED al GPIO, pata corta a una resistencia de 330Ω y luego a GND).



Optimización de Código en Ensamblador

1

Uso eficiente de registros

Mantener valores frecuentemente usados en registros evita accesos a memoria, aprovechando los 31 registros de propósito general. El compilador debe gestionar adecuadamente la asignación, priorizando variables críticas. También es importante considerar el calling convention para preservar valores entre llamadas a funciones.

2

Reducción de dependencias

Reorganizar instrucciones para minimizar dependencias de datos permite al procesador ejecutar más instrucciones en paralelo. Técnicas como loop unrolling (desenrollado de bucles) o software pipelining reducen los saltos y mejoran la utilización de la pipeline de instrucciones.

3

Uso de instrucciones SIMD

Las extensiones NEON permiten procesamiento de datos en paralelo, realizando la misma operación sobre múltiples elementos simultáneamente. Ideal para procesamiento multimedia y matemático, puede proporcionar aceleraciones significativas en algoritmos que manipulan grandes conjuntos de datos.

4

Optimización de accesos a memoria

Alinear datos para accesos más eficientes, utilizar instrucciones de transferencia múltiple (LDM/STM), y ordenar accesos para aprovechar la localidad espacial y temporal de la caché, mejorando significativamente el rendimiento.

Depuración de Programas en Ensamblador

1

Herramientas de depuración

GDB (GNU Debugger) es la herramienta principal para depurar programas en ensamblador en Raspberry Pi. Permite establecer puntos de interrupción, examinar registros y memoria, ejecutar código paso a paso y monitorear cambios. Extensiones como GEF (GDB Enhanced Features) proporcionan interfaces más amigables visualizando registros y memoria de forma más clara.

2

Puntos de interrupción

Los puntos de interrupción (breakpoints) detienen la ejecución en puntos específicos del código. En ARM, GDB implementa estos puntos sustituyendo temporalmente la instrucción original por una que genera una excepción de depuración. Pueden establecerse condicionalmente para activarse solo cuando se cumplan ciertas condiciones.

3

Análisis de registros y memoria

Durante la depuración, podemos examinar el contenido de registros (info registers), ver regiones de memoria (x/10wx dirección) o modificar valores (set $x0 = 5). Para programas que interactúan con hardware, es crucial examinar el estado de los registros mapeados en memoria para verificar la correcta configuración de periféricos.

UCO
Universidad Católica de Oriente
Vigilada Mineducación

Renovación Acreditación Institucional en ALTA CALIDAD



¡Gracias!